IACR Transactions on Symmetric Cryptology
ISSN XXXX-XXXX, Vol. 0, No. 0, pp. 1–24. DOI:XXXXXXXX

Cryptanalysis of ChiLow with Cube-Like Attacks*

Shuo Peng[1], Jiahui He[1], Kai Hu[1], Zhongfeng Niu[2], Shahram Rasoolzadeh[3] and Meiqin Wang[1]

[1] School of Cyber Science and Technology, Shandong University, Qingdao, China.
pengshuo@mail.sdu.edu.cn, qq591106627@126.com, kai.hu@sdu.edu.cn, mqwang@sdu.edu.cn
[2] School of Cryptology, University of Chinese Academy of Sciences, Beijing, China.
niuzhongfeng1996@163.com
[3] Ruhr University Bochum, Bochum, Germany. shahram.rasoolzadeh@rub.de

Abstract. Proposed in EUROCRYPT 2025, ChiLow is a family of tweakable block ciphers and a related PRF built on the novel nonlinear χ function, designed to enable efficient and secure embedded code encryption. The only key-recovery results of ChiLow are from designers which can reach at most 4 out of 8 rounds, which is not enough for a low-latency cipher like ChiLow: more cryptanalysis efforts are expected. Considering the low-degree χ function, we present three kinds of cube-like attacks on ChiLow-32 under both single-tweak and multi-tweak settings, including

- a *conditional cube attack* in the multi-tweak setting, which enables full key recovery for 5-round and 6-round instances with time complexities 2^{32} and 2^{120}, data complexities $2^{23.58}$ and 2^{40}, and negligible memory requirements, respectively.
- a *borderline cube attack* in the multi-tweak setting, which recovers the full key of 5-round ChiLow-32 with time, data, and memory complexities of 2^{32}, $2^{18.58}$, and $2^{33.56}$, respectively. For 6-round ChiLow-32, it achieves full key recovery with time, data, and memory complexities of 2^{34}, $2^{33.58}$, and $2^{54.28}$, respectively. Both attacks are practical.
- an *integral attack* on 7-round ChiLow-32 in the single-tweak setting. By combining a 4-round borderline cube with three additional rounds, we reduce the round-key search space from 2^{96} to 2^{73}. Moreover, we present a method to recover the master key based on round-key information, allowing us to recover the master key for 7-round ChiLow-32 with a time complexity of $2^{127.78}$.

All of our attacks respect security claims made by the designers. Though our analysis does not compromise the security of the full 8-round ChiLow, we hope that our results offer valuable insights into its security properties.

Keywords: ChiLow · ChiChi · Key Recovery · Cube Attack · Conditional Cube · Borderline Cube · Intergral Attack

1 Introduction

With the proliferation of diverse cryptographic and authentication requirements, a growing number of context-specific authenticated encryption algorithms have been developed. Among these, the ChiLow low-latency cipher [BDG+25] stands as one of the most recent representative designs. Tailored to ensure code confidentiality in external memory for microprocessors, ChiLow achieves exceptional low-latency performance – critical as every

*This work has been merged with ePrint paper 2025/1806 and revised into the new ePrint paper 2026/365.

instruction must be decrypted before execution. Implemented using the Nangate 15nm Open Cell Library, it demonstrates a decryption latency of under 280 picoseconds.

Two philosophical approaches underpin confidence in cryptographic algorithm security. The first employs security reduction, wherein the security of a primitive is formally reduced to the hardness of a well-studied problem. For example: The second approach relies on cryptanalysis – sustained analytical efforts to identify vulnerabilities. A prominent example is the Advanced Encryption Standard (AES) [DR20], which remains secure after over two decades of intensive global cryptanalysis.

Regarding the security of the ChiLow cipher specifically, only the second approach (cryptanalysis) is applicable. Similar to other ultra-low-latency algorithms, ChiLow's design intentionally deviates from conventional cryptographic paradigms to achieve extreme latency optimization. This unconventional architecture is mainly characterized by its extremely small 32-bit block size and short 8 rounds.

Although the designers have conducted security analyses of ChiLow, the second philosophical approach necessitates further scrutiny. Given that ultra-low-latency algorithms operate with deliberately minimized security margins, such external validation becomes particularly critical. Here we briefly recall the existing security results from the designers. The designers considered differential and linear cryptanalysis, as well as their clustering effects. They concluded that 3 rounds are already strong enough to resist the differential and linear attacks. In terms of the algebraic attacks, the designers checked that there should not be any integral distinguishers for more than 5 rounds of ChiLow. For key recovery, the designers employed a Meet-in-the-Middle attack; however, it is effective only for a 4-round key recovery.

Surprisingly, cube attacks are not considered by designers, even though the only nonlinear component of ChiLow, i.e., χ, has a low degree of 2. The integral cryptanalysis cannot completely reflect a cipher's strength against cube-like attacks. It is necessary to take the cube-like attacks into consideration to strengthen our confidence on ChiLow.

The cube attack was proposed by Dinur and Shamir [DS09] and applied to stream ciphers, as well as permutation-based ciphers such as Ascon [DEMS21]. At Eurocrypt 2017 [HWX+17], the conditional cube attacks were used to attack Keccak keyed mode ciphers. Now, the cube attack has been one of the mainstream techniques to test the security of low-degree ciphers.

Contributions. The χ function in ChiLow has a low algebraic degree of 2, where each degree-2 monomial involves the product of only two adjacent variables. Based on the properties of χ, we construct cube distinguishers for up to 6 rounds of ChiLow using a cube of dimension $2^{r-1}+1$, by carefully selecting cube variables that are non-adjacent. Furthermore, we show that ChiLow exhibits a borderline cube property, in which the corresponding superpoly depends on only a few key bits. If the cube variables are chosen to be nonadjacent with dimension 2^{r-1} for $r \leq 6$, the dependency of cube sums on a key bit depends on whether the bit is multiplied with the cube variables in the first round. Leveraging both the cube distinguisher and the borderline cube property, this paper presents three types of cube-like attacks.

- **Conditional Cube Attack: with negligible memory complexity.** We conduct conditional cube attacks in the multi-tweak setting, where cube variables are selected from both the data path and the tweak path. By exploiting the algebraic properties of χ, we find cube distinguishers for 5-round and 6-round ChiLow-32 with complexities of 2^{17} and 2^{33}, respectively, by choosing nonadjacent cube variables. We then observe that if two cube variables are adjacent to each other, the degree with respect to the cube variables increases to 17 and 33 for the 5-round and 6-round cases. This increase arises from the presence of degree-2 monomials in the first round and degree-3 monomials in the second round. Crucially, the degree-3 monomials can be

eliminated under specific conditions on the secret key, which enables the recovery of key information. As a consequence, we achieve full key recovery on 5-round ChiLow-32 with a time complexity of 2^{32}, a data complexity of $2^{23.58}$, and negligible memory requirements. For 6-round ChiLow-32, we give a full key recovery with a time complexity of 2^{120}, a data complexity of 2^{40}, and negligible memory requirements.

- **Borderline Cube Attack: practical time complexity.** We also conduct borderline cube attacks on ChiLow-32 in the multi-tweak setting. By leveraging this borderline cube property in ChiLow, we apply borderline cube attacks on both 5-round and 6-round ChiLow-32. For the 5-round case, we employ a 16-dimensional cube with nonadjacent cube variables. Since the corresponding superpoly involves only a small number of key bits, recovering the superpoly allows us to extract the key bits. Consequently, we achieve full key recovery for 5-round ChiLow with a time complexity of 2^{32}, a data complexity of $2^{18.58}$, and a memory complexity of $2^{33.56}$ bits. For the 6-round case, we employ a 32-dimensional cube with non-adjacent cube variables, achieving full key recovery with a time complexity of 2^{34}, a data complexity of $2^{33.58}$, and a memory complexity of $2^{54.28}$ bits. Both key recoveries of 5 rounds and 6 rounds are practical attacks.

- **Integral Attack: key recovery for 7 rounds.** We further propose a method to recover the round keys for 7-round ChiLow-32 in the single-tweak setting, based on the integral attack. The attack leverages a 4-round borderline cube of dimension 8, to which three additional rounds are appended. According to the borderline cube property, the resulting superpoly involves only 9 key bits. We then encrypt the corresponding 2^8 plaintexts through the last three rounds while guessing the round keys of the 5th, 6th, and 7th rounds (96 bits in total). This yields an overdefined system of equations with 9 variables and 32 equations, allowing the reduction of the round key search space from 2^{96} to 2^{73}. Due to the nested tweak-key schedule in ChiLow, recovering the master key directly from the round key information is challenging. We give a straightforward method to recover the master key from the obtained round key information with a time complexity of $2^{127.78}$, and we highlight the problem of efficiently recovering the master key as an open research question.

All of our attacks adhere to the security claim made by the designer, which states that in the single-key setting the data cannot exceed 2^{40}, and in the single-tweak setting, the data cannot exceed 2^8. A comparison of our attacks is shown in Table 1.

Outline This paper is organized as follows. Section 2 introduces the notation and definitions used throughout the paper. Section 3 presents the specification of ChiLow and and some insights to ChiLow. In Section 4, we describe conditional cube attacks on 5- and 6-round ChiLow-32. Section 5 illustrates our borderline cube attacks on 5- and 6-round ChiLow-32. Section 6 illustrates the integral attack of recovering the round key for 7-round ChiLow-32. Finally, Section 7 concludes the paper. The experiment codes and results reported in this paper are publicly available at https://github.com/chilowcube/Cryptanalysis-of-chilow-with-Cube-Like-Attacks.

2 Preliminaries

2.1 Notations

We use $\mathbb{F}_2$ to represent the finite field with two elements, namely $\{0, 1\}$, with 0 and 1 respectively being the identity elements for addition and multiplication in the field. For any positive integer n, we use $\mathbb{F}_2^n$ to denote the n-dimensional vector space over $\mathbb{F}_2$. Besides, we use $\mathbb{Z}_n$ to denote the set of non-negative integers smaller than n, i.e., $\{0, 1, \ldots, n-1\}$. For

Table 1: Summary of attacks on ChiLow. The "Setting" column lists the attack setting, including single-tweak (ST) and multi-tweak (MT). In the "Method" column, "Dif." is short for differential, "Lin" for linear, "Mitm" for meet-in-the-middle, "Con" for conditional, "Bord" for borderline, and "Int" for integral attacks.

Attack Type	Cipher	Setting	#R	T/D/M	Method	Source
Distinguisher	Both	ST	3/8	$2^{19}/2^{19}/0$	Dif	[BDG+25]
	ChiLow-32	MT	3/8	$2^{26}/2^{26}/0$	Dif	[BDG+25]
	ChiLow-40	MT	3/8	$2^{30}/2^{30}/0$	Dif	[BDG+25]
	ChiLow-32	ST	3/8	$2^{28}/2^{28}/0$	Lin	[BDG+25]
	ChiLow-40	ST	3/8	$2^{32}/2^{32}/0$	Lin	[BDG+25]
	Both	MT	5/8	$2^{17}/2^{17}/0$	Cube	Section 3
	Both	MT	6/8	$2^{33}/2^{33}/0$	Cube	Section 3
Key recovery	Both	ST	3/8	2^{126}	Mitm	[BDG+25]
	Both	ST	4/8	2^{102}	Mitm	[BDG+25]
	ChiLow-32	MT	5/8	$2^{32}/2^{23.58}/0$	Con.cube	Section 4
	ChiLow-32	MT	6/8	$2^{120}/2^{40}/0$	Con.cube	Section 4
	ChiLow-32	MT	5/8	$2^{32}/2^{18.58}/2^{33.56}$	Bord.cube	Section 5
	ChiLow-32	MT	6/8	$2^{34}/2^{33.58}/2^{54.28}$	Bord.cube	Section 5
	ChiLow-32	ST	7/8	$2^{127.78}/2^{8}/2^{23}$	Int	Section 6

an n-bit vector $u = (u[0], \ldots, u[n-1]) \in \mathbb{F}_2^n$, we denote by $u[i:j]$ the subvector consisting of the bits from position i to position j of u. The Hamming weight of u is defined as $\mathrm{wt}(u) = \sum_{i=0}^{n-1} u[i]$, where each $u[i]$ is interpreted as an integer.

Boolean Function. Let $f : \mathbb{F}_2^n \to \mathbb{F}_2$ be a Boolean function whose ANF is

$$f(x) = f(x[0], x[1], \ldots, x[n-1]) = \sum_{u \in \mathbb{F}_2^n} a_u x^u = \sum_{u \in \mathbb{F}_2^n} a_u \prod_{i=0}^{n-1} x[i]^{u[i]},$$

where $a_u \in \mathbb{F}_2$, and

$$x^u = \pi_u(x) = \prod_{i=0}^{n-1} x[i]^{u[i]} \text{ with } x[i]^{u[i]} = \begin{cases} x[i], & \text{if } u[i] = 1, \\ 1, & \text{if } u[i] = 0, \end{cases}$$

is called a monomial. For each $u \in \mathbb{F}_2^n$, the coefficient $\alpha_u \in \mathbb{F}_2$ is of the monomial x^u in f that we denote by $\mathrm{Coeff}_f(x^u)$.

Keyed Boolean Functions. It is often necessary to distinguish controllable public input variables from inaccessible secret input variables. We denote the public variables by x and the secret variables by k. With such a distinction, when a Boolean function depends on n public variables x and m key variables k, we look at it as

$$f(x, k) = \sum_{u \in \mathbb{F}_2^n} \alpha_u(k)\ x^u\,. \tag{1}$$

For simplicity, we also denote such a keyed Boolean function as $f_k(x)$. Note that in this equation, the coefficient $\mathrm{Coeff}_{f_k}(x^u)$ is actually a Boolean function of the form $\mathbb{F}_2^m \to \mathbb{F}_2$ mapping k to $\alpha_u(k)$. When we refer to the degree of a keyed Boolean function, we mean the degree in public variables:

$$\deg(f) := \max\{\mathrm{wt}(u) \mid \forall u \in \mathbb{F}_2^n \text{ with } \alpha_u \text{ not being the constant-zero function }\}\,.$$

2.2 Cube Attack

The cube attack was proposed at EUROCRYPT 2009 by Dinur and Shamir to analyze black-box tweakable polynomials [DS09]. Let $I \subseteq \mathbb{Z}_n$, with $\bar{I}$ its complement and $|I|$ its size. For variables $x = (x[0], \ldots, x[n-1])$, define $x[I] = \{x[i] : i \in I\}$ and $x^I = \prod_{i \in I} x[i]$. The keyed Boolean function in Equation 1 can then be written as

$$f(x,k) = x^I \cdot p_I(x[\bar{I}], k) + q(x,k),$$

where each term of $q(x,k)$ omits at least one variable from $x[I]$.

We call x^I the *cube term* or *cube monomial* and $p_I(x[\bar{I}], k)$ the *superpoly* of x^I in $f(x,k)$. If we set the variables in $x[\bar{I}]$ to some fixed constants, the superpoly $p_I(x[\bar{I}], k)$ is a Boolean function of k. Concerning the superpoly, we have the following lemma.

Lemma 1 ([DS09]). *For a set $I \subseteq \{0, \ldots, n-1\}$ and a keyed Boolean function $f(x,k)$ with*

$$f(x,k) = \sum_{u \in \mathbb{F}_2^n} a_u(k) x^u = x^I \cdot p_I(x[\bar{I}], k) + q(x,k),$$

we have

$$p_I(x[\bar{I}], k) = \sum_{x[I] \in \mathbb{F}_2^{|I|}} f(x,k).$$

In Lemma 1, if $I = \mathbb{Z}_n$, then the superpoly of x^I in $f(x,k)$ is equal to $\mathrm{Coeff}_f(x^I)$, which in the case of bijective functions, such as in block ciphers, is 0.

Given a monomial x^u, the following Lemmas provide a theoretical basis for recovering its superpoly.

Lemma 2 ([CCH10, Can16]). *Given an oracle access to the Boolean function f, the coefficient of the monomial x^u in f for a particular u can be computed as $\mathrm{Coeff}_f(x^u) = \sum_{x \preccurlyeq u} f(x)$ with $2^{wt(u)}$ evaluations of f.*

Lemma 3 ([CCH10, Can16]). *The set of all coefficients α_u with $u \in \mathbb{F}_2^n\}$ of the ANF can be obtained from the truth table of f with so-called fast Möbius transform with about $n \cdot 2^n$ XOR operations.*

Lemma 4 ([RHSS21]). *For the keyed Boolean function given in Equation 1, it takes $2^{m+\mathrm{wt}(u)}$ evaluations of f_k to recover $\mathrm{Coeff}_{f_k}(x^u)$ for a certain u where $\mathrm{wt}(u) > \log_2(m)$.*

If the algebraic expressions of f_k are not overly complex, symbolic computation provides an efficient approach for recovering $\mathrm{Coeff}_f(x^u)$. When the ANF or certain partial ANF of f_k are available, the corresponding superpoly can often be derived directly. In a block cipher, the encryption (or decryption) function is composed of multiple round functions, which allows $\mathrm{Coeff}_f(x^u)$ to be computed incrementally, one round at a time. An alternative strategy leverages the division property [Tod15] in combination with automated search techniques [XZBL16]. In particular, the bit-based division property [TM16] can determine the presence or absence of specific monomials in the target Boolean function. This capability makes it a powerful tool for partially or fully characterizing the algebraic structure of superpolies, which has been successfully applied in various contexts of cube attacks [TIHM17, WHT+18, WHG+19, HLM+20, HLLT20, HSWW20]. In this paper, we adopt symbolic computation as the primary method for recovering superpolys.

Since been proposed, several variants of cube attack have been proposed to enhance its ability. *Conditional Cube Attack* [LBDW17, DLWQ17, LDW17, SGSL18, SG18] extends the classical cube attack by incorporating additional conditions on the key or internal state.

Instead of evaluating the output over all possible cube values, it focuses only on inputs that satisfy specific conditions, which can reduce computational complexity or enhance the distinguishability of superpolys. This allows the attack to succeed even when classical cube attacks fail due to high-degree terms or partial cancellations. In [Hu24], a break and fix strategy is applied: the attacker first disrupts the algebraic structure (break phase) to increase the degree of output bits. Then, key conditions are applied to restore the structure (fix phase), allowing the attacker to recover the key information by observing changes in the algebraic degrees. *Borderline cube* [DMP+15, RHSS21, PHHW25] refers to a special type of cube in cube attacks whose superpoly depends only on a relatively small subset of the secret key bits. In other words, the superpoly of a borderline cube is not fully independent of the key, but it involves only a small subset of key variables. This makes borderline cubes useful for cryptanalysis, as they reduce the effective key space and often serve as a basis for distinguishers or partial key-recovery attacks.

3 ChiLow

3.1 Specification

ChiLow [BDG+25] is a family of tweakable block ciphers and a related PRF for embedded code encryption. The main ciphers, ChiLow-32 and ChiLow-40, follow a standard SPN structure. ChiLow-32 works on 32-bit blocks with a 128-bit key and a 64-bit tweak. Its core primitive, D_{32}, decrypts 32-bit instructions. A derived function, $D'_{32}(K, T, X)$, truncated to $\tau \in 8, 16$ bits, provides authentication tags. An illustration of ChiLow-32 is shown in Figure 1. ChiLow-40 operates on 40-bit blocks with the same key and tweak sizes. Its primitive, D_{40}, maps 40-bit inputs to outputs, forcing the last 8 bits to zero. This design combines decryption and authentication in one step.

The construction of these tweakable block ciphers relies on three interacting states: a 128-bit key state updated by round-dependent permutations (key schedule), a 64-bit tweak state updated by a family of tweak-dependent permutations (tweak schedule), and a cipher state of 32 or 40 bits (for D_{32}/D'_{32} or D_{40}). Each permutation in the key schedule, tweak schedule, and data path consists of a nonlinear layer $\mathcal{X}$ followed by a linear layer L. Their sizes are 32 bits for D_{32} ($\mathcal{X}_{32}, L_{32}$) and D'_{32} ($\mathcal{X}_{32}, L'_{32}$), 40 bits for D_{40} ($\mathcal{X}_{40}, L_{40}$), 64 bits for the tweak schedule ($\mathcal{X}_{64}, L_{64}$), and 128 bits for the key schedule ($\mathcal{X}_{128}, L_{128}$). Round layers are denoted $\mathcal{X}^{(i)}$ and $L^{(i)}$. In the data path, $X_i^{\oplus}/X_i^{\dagger}$ and in the tweak schedule, $T_i^{\oplus}/T_i^{\dagger}$ denote the states before/after $\mathcal{X}$.

Nonlinear layer: $\mathcal{X}$. The $\mathcal{X}$ function is a bijective mapping derived from the well-known χ function. For an integer n, the χ function is defined as

$$\chi_n : \mathbb{F}_2^n \to \mathbb{F}_2^n, \quad y_i = x_i + (x_{i+1} + 1)x_{i+2},$$

where the indices are taken modulo n. The nonlinear function $\mathcal{X}$ can be viewed as an intertwined variant of two parallel χ functions, and is formally defined as

$$\mathcal{X}_n(x) = \begin{pmatrix} \chi_{m-1}(x[0], x[1], \ldots, x_{[}m-1]) \\ \chi_{m+1}(x[m], x_{[}m+1], \ldots, x_{[}2m-1]) \end{pmatrix} + \lambda(x),$$

where m is even, $n = 2m$, and the i-th coordinate of λ is given by

$$\lambda_i(x) = \begin{cases} x_m + x_{m-3}, & \text{if } i = m-3, \\ x_{m-1} + x_{m-2}, & \text{if } i = m-2, \\ x_{m-3} + x_m + x_{m-1}, & \text{if } i = m-1, \\ x_m + x_{m-2}, & \text{if } i = m, \\ 0, & \text{otherwise.} \end{cases}$$

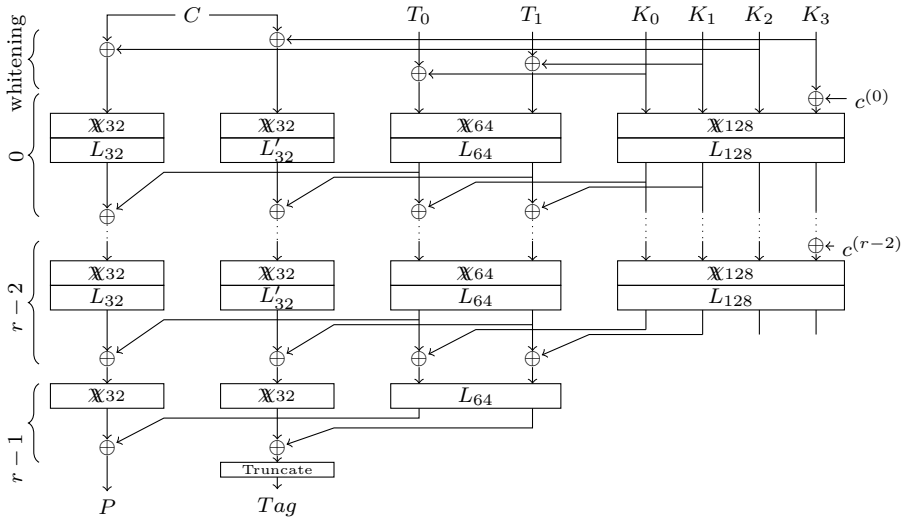


Figure 1: A full ChiLow-32 decryption and tag computation taken from [BDG+25]. Wires represent 32-bit values.

Linear layer: L**.** The linear mappings L_{128}, L_{64}, L_{40}, L_{32} and L'_{32} all have the same structure. Let $y = L(x)$ with $x, y \in \mathbb{F}_2^{128}$ or $\mathbb{F}_2^{64}$ or $\mathbb{F}_2^{40}$ or $\mathbb{F}_2^{32}$. Then,

$$y_i = x_{\alpha i+\beta_0} + x_{\alpha i+\beta_1} + x_{\alpha i+\beta_2},$$

where the indices must be computed modulo the vector size. The offsets used in the linear maps are as follows: for L_{32}, $\alpha = 11$, $\beta_0 = 5$, $\beta_1 = 9$, $\beta_2 = 12$; for L'_{32}, $\alpha = 11$, $\beta_0 = 1$, $\beta_1 = 26$, $\beta_2 = 30$; for L_{40}, $\alpha = 17$, $\beta_0 = 1$, $\beta_1 = 9$, $\beta_2 = 30$; for L_{64}, $\alpha = 3$, $\beta_0 = 1$, $\beta_1 = 26$, $\beta_2 = 50$; and for L_{128}, $\alpha = 17$, $\beta_0 = 7$, $\beta_1 = 11$, $\beta_2 = 14$.

Key schedule. The key schedule maintains a 128-bit state, which is initialized with the master key and updated in each round by the key schedule's round function. The round function consists of round-constant addition, a nonlinear mapping ($\mathfrak{X}_{128}$), and a linear diffusion layer(L_{128}), and is defined as

$$K_{i+1} = L_{128}^{(i)}\big(\mathfrak{X}_{128}^{(i)}(K_i \oplus c^{(i)})\big).$$

Tweak schedule. The tweak schedule uses a 64-bit state T_i that is initialized with the input tweak and whitened with key material, namely, $T_0 = T \oplus K[0:63]$. Then, the state is updated with the round function. The round function consists of round-constant addition, a nonlinear mapping ($\mathfrak{X}_{64}$), and a linear diffusion layer(L_{64}), and is defined as

$$T_{i+1} = L_{64}^{(i)}\big(\mathfrak{X}_{64}^{(i)}(T_i)\big) \oplus K_{i+1}[0:63],$$

The last round is composed only of the linear diffusion layer, namely,

$$T_r = L_{64}^{(r-1)}(T_{r-1}).$$

Data Path. For ChiLow-32, the data path uses a 32-bit state $X_{(i)}$, which is initialized with the input block and whitened with key material. Specifically, for D_{32}, the initial state is $X_{(0)} = X \oplus K_0[64:95]$, and for D'_{32}, it is $X_{(0)} = X \oplus K_0[96:127]$, Subsequently, the state is updated using the round function. The round function for D_{32} is as follows:

$$X_{i+1} = L_{32}^{(i)}\big(\mathcal{X}_{32}^{(i)}(X_i)\big) \oplus T_{i+1}[0:31], \quad \text{for } i \leq \mathrm{r}-2,$$
$$X_r = \mathcal{X}_{32}^{(r-1)}(X_{r-1}) \oplus T_r[0:31].$$

The round function of D'_{32} is very similar to that of D_{32}, but it uses a different linear mapping and takes a different part of the tweak state:

$$X'_{i+1} = L'^{(i)}_{32}\big(\mathcal{X}_{32}^{(i)}(X'_i)\big) \oplus T_{i+1}[32:63], \quad \text{for } i \leq \mathrm{r}-2,$$
$$X'_r = \mathcal{X}_{32}^{(r-1)}(X'_{r-1} \oplus T_r[32:63].$$

The output of D'_{32} is obtained by truncating X'_r to the first τ bits, i.e., $X'_r[0:\tau-1]$.

InChiLow-40, the data path uses a 40-bit state X_i and 40-bit mappings. It is initialized with the input block and whitened with key material, namely, $X_0 = X \oplus K_0[64:103]$. For the rest, D_{40} is very similar to D_{32} and D'_{32}:

$$X_{i+1} = L_{40}^{(i)}\big(\mathcal{X}_{40}^{(i)}(X_i)\big) \oplus T_{i+1}[0:39], \quad \text{for } i \leq \mathrm{r}-2,$$
$$X_r = \mathcal{X}_{40}(X_{r-1} \oplus T_r[0:39].$$

The output block is expected to contain the 32-bit instruction followed by 8 bits equal to zero.

3.2 Insights into ChiLow

3.2.1 Understanding the Security Claim and Cube Distinguishers

In [BDG+25], the designers made the following security claims for ChiLow,

1. the number of queries under the same tweak should not exceed 2^8.
2. the total number of queries should remain below 2^{40}.

Thus, two types of selecting cube variables are possible in the single-key cube attack,

1. In the single-tweak setting, the cube variables must be chosen solely from the data path and are limited to a maximum of 8 bits.
2. In the multi-tweak setting, let d and t be the numbers of cube variables selected in the data and tweak paths, respectively; we have $d \leq 8$ and $d + t \leq 40$.

ChiLow uses the 2-degree $\mathcal{X}$ function as its nonlinear layer in every round function. Thus, the upper bound on the algebraic degrees of ChiLow output bits after r rounds should be 2^r ($r \leq 6$). However, since the quadratic terms in the $\mathcal{X}$ function arise only from multiplying adjacent cube variables, if we judiciously select non-adjacent input bits as cube variables while fixing other bits as constants, the algebraic degree of the first-round output variables will remain 1. Consequently, the upper bound of the algebraic degree for the r-round ChiLow output bits will be 2^{r-1} ($r \leq 6$).

Recall that $\mathcal{X}$ is defined as

$$\mathcal{X}_n(x) = \begin{pmatrix} \chi_{m-1}(x_0, x_1, \ldots, x_{m-1}) \\ \chi_{m+1}(x_m, x_{m+1}, \ldots, x_{2m-1}) \end{pmatrix} + \lambda(x),$$

where m is even, $n = 2m$, and $\lambda(x)$ is a linear function of the input vector x. Given χ_p where p is an odd number, at most $(p-1)/2$ non-adjacent input variables can be selected as the cube variables. Thus, we can select up to $m-1$ cube variables for the input of $\mathcal{X}$.

ChiLow-32 uses the 32-bit and 64-bit $\mathcal{X}$ for the its data and tweak paths. The 64-bit tweak path allows for up to 31 non-adjacent cube variables. The 32-bit data part permits up to 15 non-adjacent cube variables. In total, 47 cube variables can be selected, which is sufficient to construct a distinguisher for up to 6 rounds of ChiLow-32 in the multi-tweak model.

3.2.2 Borderline Cube in ChiLow

Recall that the quadratic terms of χ arise only from products of adjacent input variables, choosing non-adjacent bits as cube variables (while fixing the others) keeps the first-round degree at 1. As a result, since the degree of the round function is 2, the algebraic degree of the r-round ChiLow output bits is bounded by 2^{r-1} ($r \leq 6$). When choose a 2^{r-1} dimensions cube that cube variables are not adjacent, if a key bit is not multiplied with the cube variables in the first round, then the cube sums do not depend on the value of this bit. On the other hand, if a key bit is multiplied with the cube variables in the first round, then the cube sums generally depend on the value of the bit.

In the first round, the multiplication between cube variables and key bits arises solely from the χ function. Each cube variable multiplies with at most two key bits. For example, the cube variable $T_0[32]$ in the first-round χ function clearly demonstrates this property.

$$\begin{aligned}
T_0^{\dagger}[31] &= T_0^{\oplus}[31] + (T_0^{\oplus}[32] + 1)T_0^{\oplus}[33] \\
&= T_0[31] + K_0[31] + (T_0[32] + K_0[32] + 1)(T_0[33] + K_0[33]) \\
&= T_0[31] + K_0[31] + T_0[32]T_0[33] + T_0[32]K_0[33] \\
&\quad + K_0[32]T_0[33] + K_0[32]K_0[33] + T_0[33] + K_0[33], \\
T_0^{\dagger}[63] &= T_0^{\oplus}[63] + (T_0^{\oplus}[31] + 1)T_0^{\oplus}[32] \\
&= T_0[63] + K_0[63] + (T_0[31] + K_0[31] + 1)(T_0[32] + K_0[32]) \\
&= T_0[63] + K_0[63] + T_0[31]T_0[32] + T_0[31]K_0[32] + K_0[31]T_0[32] \\
&\quad + K_0[31]K_0[32] + T_0[32] + K_0[32].
\end{aligned}$$

Other cube variables may multiply with the same key bit. For instance, $T_0[34]$ also multiplies with $K_0[33]$. Therefore, if we select 2^{r-1} ($r \leq 6$) non-adjacent cube variables for r rounds of ChiLow, the corresponding superpoly depends on at most $2 \cdot 2^{r-1}$ key bits (possibly fewer when duplicate key bits are involved).

This gives rise to a borderline cube situation. Specifically, for 4 rounds, an 8-dimensional non-adjacent cube involves at most 16 key bits; for 5 rounds, a 16-dimensional non-adjacent cube involves at most 32 key bits; and for 6 rounds, a 32-dimensional non-adjacent cube involves at most 64 key bits. Hence, given the lenth of secret key is 128, borderline cubes can be constructed for 4-, 5-, and 6-round ChiLow.

4 Conditional Cube Attack of ChiLow-32

4.1 Practical Attack on 5-round ChiLow-32

We apply the conditional cube attack with the *break-fix* to 5-round ChiLow-32 and present a practical attack. The break-fix strategy consists of two phases. The *break* phase deliberately modifies the cube to increase output degrees, and the *fix* phase imposes key-dependent conditions to restore them. By checking whether the degrees return to their original values, the attacker can extract key information.

Distinguishers and cube configuration. In this attack, we use 17 cube variables that lead to a zero-sum for the 5-round ChiLow-32 outputs. Without loss of generality, we select 8 cube variables from the data path and 9 cube variables from the tweak path, which we denote as Cube-I, specified as follows:

$$\begin{aligned}
\text{Data path: } &\{16, 18, 20, 22, 24, 26, 28, 30\}, \\
\text{Tweak path: } &\{23, 32, 34, 40, 48, 50, 55, 57, 62\}.
\end{aligned}$$

Break phase. In the *break* phase, we modify the cube structure by replacing the 62-nd bit with the 31-st bit in the tweak path, resulting in two adjacent bits within the cube.

The modified cube, denoted as Cube-II, is defined as follows:

$$\begin{aligned}\text{Data path: } &\{16, 18, 20, 22, 24, 26, 28, 30\},\\ \text{Tweak path: } &\{23, 31, 32, 34, 40, 48, 50, 55, 57\}.\end{aligned}$$

For Cube-I, the degree (w.r.t. the cube variables) of all output bits after 5 rounds is at most 16, whereas under Cube-II it rises to 17. The increase in degree is primarily caused by the adjacency of cube variables introduced by the modification. In the following, we analyze the first two rounds to illustrate the impact of these two adjacent bits.

A comparison between Cube-I and Cube-II is shown in Figure 2. In Figure 2a, under Cube-I, since the cube variables are not adjacent, the degree after the first round is 1, after the second round is 2, and after five rounds the upper bound of the degree reaches 16. For Cube-II, let's focus on the tweak path in Figure 2b. In the first round, according to the ANFs of $\mathfrak{X}_{64}$, the 63-rd output bit of $\mathfrak{X}_{64}^{(0)}$(The ■ bit in $T_0^\dagger$ in Figure 2b) has degree 2, as

$$\begin{aligned}T_0^\dagger[63] &= T_0^\oplus[63] + (T_0^\oplus[31] + 1)T_0^\oplus[32]\\ &= (T_0[63] + K_0[63]) + (T_0[31] + K_0[31] + 1)(T_0[32] + K_0[32])\\ &= T_0[31]T_0[32] + T_0[31]K_0[32] + K_0[31]T_0[32] + K_0[31]K_0[32]\\ &\quad + T_0[63] + K_0[63] + T_0[32] + K_0[32].\end{aligned}$$

Both $T_0[31]$ and $T_0[32]$ are cube variables, which causes $T_0^\dagger[63]$ to become a quadratic bit. However, since the quadratic term $T_0[31]T_0[32]$ in $T_0^\dagger[63]$ has a coefficient of 1, we cannot impose key conditions to cancel this term. To apply the conditional cube attack, we need to analyze the bits after $\mathfrak{X}$ in the second round ($\mathfrak{X}_{64}^{(1)}$).

After $L_{64}^{(0)}$, this degree-2 bit can be contained by $T_1[42], T_1[47], T_1[55]$(the three ■ bits in T_1 in Figure 2b), as

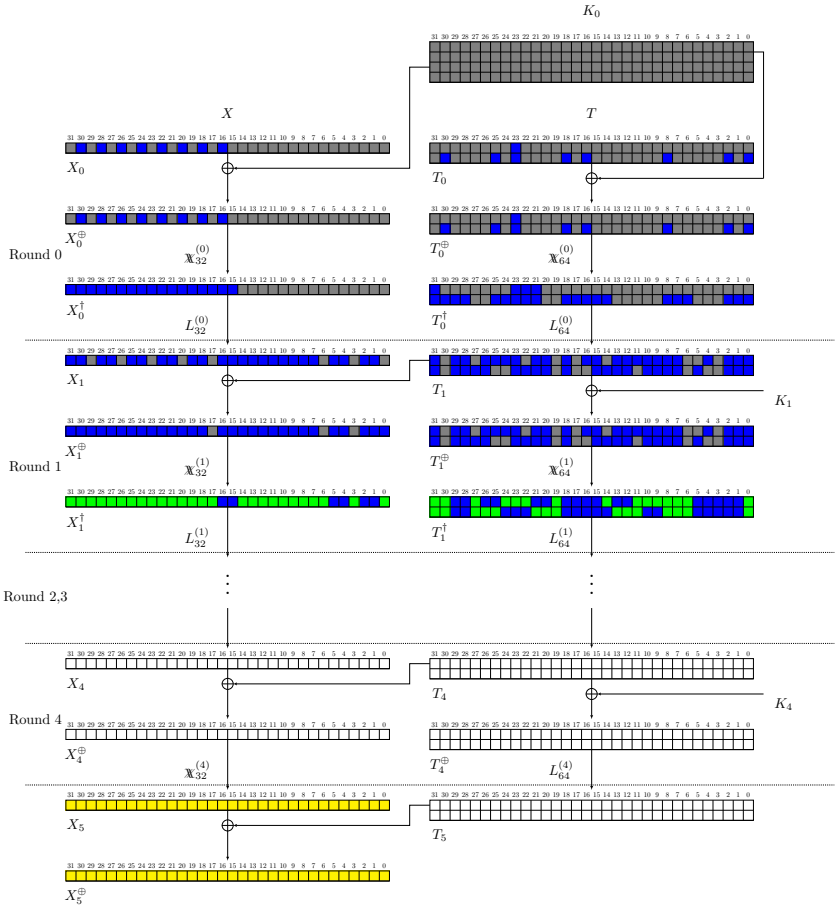
$$\begin{aligned}T_1[42] &= T_0^\dagger[63] + T_0^\dagger[24] + T_0^\dagger[48],\\ T_1[47] &= T_0^\dagger[14] + T_0^\dagger[39] + T_0^\dagger[63],\\ T_1[55] &= T_0^\dagger[38] + T_0^\dagger[63] + T_0^\dagger[23].\end{aligned}$$

The addition of key does not affect the quadratic terms; Thus, $T_1^\oplus[42], T_1^\oplus[47], T_1^\oplus[55]$(The ■ bit in $T_1^\oplus$ in Figure 2b) still contain the term $T_0[31]T_0[32]$. After $\mathfrak{X}_{64}^{(1)}$, $T_1^\oplus[42]$ is multiplied by $T_1^\oplus[41]$ and $T_1^\oplus[43]$; $T_1^\oplus[47]$ is multiplied by $T_1^\oplus[46]$ and $T_1^\oplus[48]$; and $T_1^\oplus[55]$ is multiplied by $T_1^\oplus[54]$ and $T_1^\oplus[56]$. The ANFs of $T_1^\oplus[41]$, $T_1^\oplus[43]$, $T_1^\oplus[46]$, $T_1^\oplus[48]$, $T_1^\oplus[54]$, and $T_1^\oplus[56]$ are as follows:

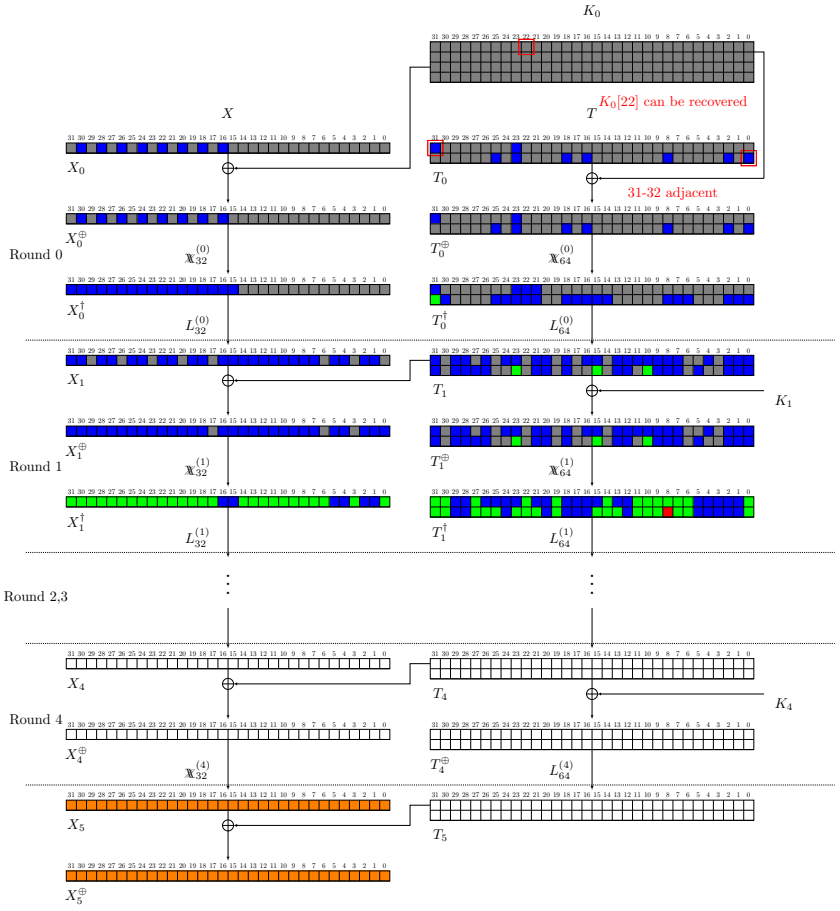
$$\begin{aligned}T_1^\oplus[41] &= T_0^\dagger[60] + K_1[60] + T_0^\dagger[21] + K_1[21] + T_0^\dagger[45] + K_1[45],\\ T_1^\oplus[43] &= T_0^\dagger[2] + K_1[2] + T_0^\dagger[27] + K_1[27] + T_0^\dagger[51] + K_1[51],\\ T_1^\oplus[46] &= T_0^\dagger[11] + K_1[11] + T_0^\dagger[36] + K_1[36] + T_0^\dagger[60] + K_1[60],\\ T_1^\oplus[48] &= T_0^\dagger[17] + K_1[17] + T_0^\dagger[42] + K_1[42] + T_0^\dagger[2] + K_1[2],\\ T_1^\oplus[54] &= T_0^\dagger[35] + K_1[35] + T_0^\dagger[60] + K_1[60] + T_0^\dagger[20] + K_1[20],\\ T_1^\oplus[56] &= T_0^\dagger[41] + K_1[41] + T_0^\dagger[2] + K_1[2] + T_0^\dagger[26] + K_1[26].\end{aligned}$$

Excluding the bits of K_1 (which are constants and do not affect the degree), the monomial $T_0[31]T_0[32]$ is multiplied by the following terms:

$$\begin{aligned}&T_0^\dagger[60], T_0^\dagger[21], T_0^\dagger[45], T_0^\dagger[2], T_0^\dagger[27], T_0^\dagger[51], T_0^\dagger[11],\\ &T_0^\dagger[36], T_0^\dagger[17], T_0^\dagger[42], T_0^\dagger[35], T_0^\dagger[20], T_0^\dagger[41], T_0^\dagger[26].\end{aligned}$$



(a) Growth of the degree in Cube-I



(b) Growth of the degree in Cube-II

Figure 2: Comparison of Cube-I and Cube-II in 5-round ChiLow-32

Algorithm 1: Recovery of the secret key bit $K_0[22]$

Input: Oracle of ChiLow-32
Output: Value of $K_0[22]$

Initialize key variable $K_0[22]$;
Compute the cube sum of Cube-II for all output bits;
if *all cube sums are equal to* 0 **then**
  $K_0[22] \longleftarrow T_0[22] + 1$;
  return $K_0[22]$;
end
else
  $K_0[22] \longleftarrow T_0[22]$;
  return $K_0[22]$;
end

Except for $T_0^\dagger[21]$, all other terms are constants (as indicated by ■ bit in $T_0^\dagger$ in Figure 2b). For $T_0^\dagger[21]$, whose ANF is given by

$$T_0^\dagger[21] = T_0[21] + K_0[21] + (T_0[22] + K_0[22] + 1)(T_0[23] + K_0[23]),$$

since $T_0[23]$ is a cube variable, this produces a unique degree-3 monomial (which is contained by the ■ bit in $T_1^\dagger$ in Figure 2b) as follows:

$$(T_0[22] + K_0[22] + 1)\, T_0[23]\, T_0[31]\, T_0[32].$$

Owing to the coefficient of the degree-3 monomial depending on the secret key, we can impose a condition on the key, which allows us to recover information about the secret key.

Fix phase. As describe above, the term $(T_0[22] + K_0[22] + 1)\, T_0[23]\, T_0[31]\, T_0[32]$ is the unique degree-3 monomial after the χ_{64} in the second round, contained by $T_1^\dagger[40]$ (the ■ bit in $T_1^\dagger$ in Figure 2b). This monomial necessarily contributes to each term of the superpoly of Cube-II, since the degree-17 monomial is generated by multiplying this degree-3 monomial with other terms.

Consequently, the superpoly of Cube-II in the i-th output bit can be expressed as

$$(T_0[22] + K_0[22] + 1) \cdot f[i],$$

where $f[i]$ is a complex polynomial that depends on the secret key.

If we set $(T_0[22] + K_0[22] + 1)$ to zero, the degree-3 term $T_0[23]\, T_0[31]\, T_0[32]$ will be canceled. Consequently, the coefficient of the degree-17 monomial in all output bits becomes zero, and the degree of all output bits after five rounds reverts to 16. In this case, we can establish a relationship between $T_0[22]$ and $K_0[22]$, which allows us to recover information about $K_0[22]$. Our key recovery algorithm is illustrated in Algorithm 1.

Note that the recovery of $K_0[22]$ relies on the following assumption.

Assumption 1. *If $T_0[22] + K_0[22] + 1 \neq 0$, then the 32 cube sums for Cube-II are never all zero.*

Assumption 1 can be verified by experiment. According to the borderline cube property in ChiLow, the superpolys of Cube-II only involving the following 24 key bits,

$$k_0[111], k_0[113], K_0[115], K_0[117], K_0[119], K_0[121], K_0[123], K_0[125],$$
$$K_0[127], K_0[22], K_0[24], K_0[30], K_0[31], K_0[32], K_0[33], K_0[35],$$
$$K_0[39], K_0[41], K_0[47], K_0[49], K_0[51], K_0[54], K_0[56], K_0[58].$$

In our experiments, we set $T_0[22] = K_0[22] = 0$ and exhaustively enumerated the values of the remaining 23 key bits. We then computed the cube sums under Cube-II and found that no assignment of these 23 key bits resulted in all cube sums being zero, thereby confirming the validity of Assumption 1.

Recovering the remaining key bits. using the same method but only slightly adjust the cube variables, we can recover other key bits. For example, if we replace the 23-rd bit with the 3-rd bit in the tweak path, we can recover the value of $K_0[4]$. In this case, the cube is defined as

$$\begin{aligned}\text{Data part: } &\{16, 18, 20, 22, 24, 26, 28, 30\},\\ \text{Tweak part: } &\{3, 31, 32, 34, 40, 48, 50, 55, 57\}.\end{aligned}$$

After χ_{64} in the second round, the unique degree-3 monomial that appears is

$$(T_0[4] + K_0[4])T_0[3]T_0[31]T_0[32].$$

If $T_0[4] + K_0[4]$ equals zero, then all corresponding superpolys in the output bits will evaluate to 0, allowing us to deduce the value of $K_0[4]$.

Similarly, by replacing the 23rd bit in the tweak path with one of the other bits from the following set, the resulting unique degree-3 monomial will change:

$$\{4, 12, 13, 18, 19, 28, 29, 37, 38, 44, 46, 47, 52, 53, 61, 62\}.$$

Applying the same method as described earlier, we are able to recover additional secret key bits. Specifically, the recovered key bits are

$$\begin{aligned}&k_0[3], K_0[13], K_0[12], K_0[19], K_0[18], K_0[29], K_0[28], K_0[38],\\ &K_0[37], K_0[43], K_0[47], K_0[46], K_0[53], K_0[52], K_0[62], K_0[61].\end{aligned}$$

In total, we can recover 18 bits of the secret key under the two adjacent cube variables $(T_0[31], T_0[32])$.

We can also modify the choice of adjacent cube variables. For instance, selecting $T_0[19]$ and $T_0[20]$ results in $T_0^{\dagger}[18]$ becoming a quadratic term. Through the linear function applied in the first round, this term propagates to $T_1[27]$, $T_1[32]$, and $T_1[40]$. In a manner analogous to the previous case, we are able to control the emergence of a unique degree-3 monomial, which in turn enables the recovery of the following secret key bits:

$$\begin{aligned}&K_0[2], K_0[1], K_0[8], K_0[7], K_0[17], K_0[16], K_0[22],\\ &K_0[40], K_0[41], K_0[42], K_0[48], K_0[55], K_0[57], K_0[56], K_0[63].\end{aligned}$$

Note that $T_1[27]$ is XORed into $X_1[27]$. The term $X_1[27]$ subsequently multiplies with $X_1[26]$ and $X_1[28]$, which can form a unique degree-3 monomial within the data path. These monomials, in turn, allows the recovery of the following secret key bits:

$$\begin{aligned}&K_0[96], K_0[97], K_0[99], K_0[100], K_0[103], K_0[104],\\ &K_0[105], K_0[106], K_0[121], K_0[122], K_0[125], K_0[126].\end{aligned}$$

By modifying the choice of adjacent cube variables, we can recover the secret key bits $K_0[i]$ for $0 \leq i \leq 63$ and $96 \leq i \leq 127$. The remaining 32 bits of the secret key, which are XORed into D'_{32} prior to the first round, can be recovered through an exhaustive search.

Complexity. Recovering a single key bit in our attack requires both time and data complexities of 2^{17}. In total, 96 secret key bits, namely $K_0[i]$ for $0 \leq i \leq 63$ and $96 \leq i \leq 127$, can be recovered by means of the conditional cube attack. The remaining key bits can subsequently be determined by an exhaustive search. The overall data complexity is given by

$$96 \cdot 2^{17} \approx 2^{6.58} \cdot 2^{17} = 2^{23.58},$$

while the total time complexity is

$$96 \cdot 2^{17} + 2^{32} \approx 2^{32}.$$

The memory requirements are minimal and can be disregarded.

4.2 Attack on 6-round ChiLow-32

Distinguishers and cube configuration. The conditional cube attack on 6-round ChiLow-32 is similar to the 5-round case. We use 33 cube variables for the 6-round output. For the distinguisher, the cube setting denoted as Cube-I is as follows

$$\begin{aligned} \text{Data path: } & \{6, 13, 15, 17, 19, 21, 23, 28\}, \\ \text{Tweak path: } & \{1, 4, 6, 8, 10, 12, 15, 17, 19, 21, 23, 27, 29\}, \\ & \{33, 35, 37, 42, 47, 49, 51, 53, 55, 57, 59, 61\}. \end{aligned}$$

Break phase. In the *break* phase, we replace the 23rd bit with the 3rd bit in the tweak path, creating two adjacent cube variables, $T_0[3]$ and $T_0[4]$. The resulting cube, denoted Cube-II, is:

$$\begin{aligned} \text{Data path: } & \{6, 13, 15, 17, 19, 21, 23, 28\}, \\ \text{Tweak path: } & \{1, 3, 4, 6, 8, 10, 12, 15, 17, 19, 21, 27, 29\}, \\ & \{33, 35, 37, 42, 47, 49, 51, 53, 55, 57, 59, 61\}. \end{aligned} \tag{2}$$

For Cube-I, the maximum degree (w.r.t. cube variables) of all 6-round outputs is 32. Under Cube-II, it increases to 33, primarily due to the adjacency of $T_0[3]$ and $T_0[4]$. Figure 3 compares Cube-I and Cube-II for 6-round ChiLow-32. In Figure 3a, for Cube-I, the non-adjacent cube variables yield a degree of 1 after the first round, 2 after the second round, and a maximum of 32 after six rounds. In Figure 3b, Cube-II shows a different degree progression due to the adjacency of cube variables.

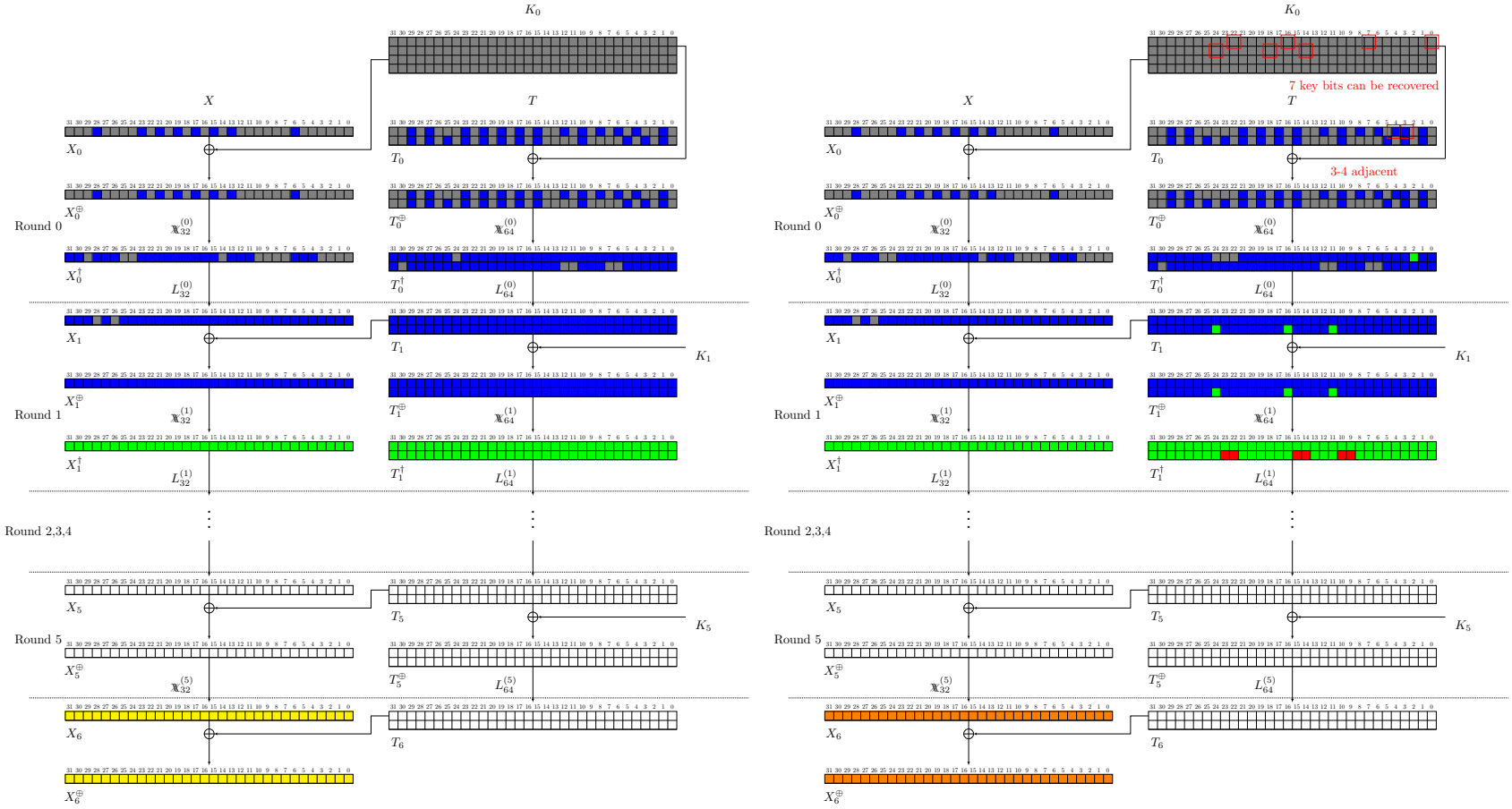
In the first round, according to the ANFs of χ_{64}, $T_0^{\dagger}[2]$ (the ■ bit in $T_0^{\dagger}$ in Figure 3b) has degree 2, containing the term $T_0[3]T_0[4]$. After the first-round linear layer (L_{64}), this degree-2 terms appears in $T_1[43]$, $T_1[48]$, and $T_1[56]$ (the three ■ bits in T_1 in Figure 3b). After the addition of key, $T_1^{\oplus}[43], T_1^{\oplus}[48], T_1^{\oplus}[56]$(The ■ bit in $T_1^{\oplus}$ in Figure 3b) still contain the term $T_0[3]T_0[4]$.

After χ_{64} in the second round $T_1^{\oplus}[43]$ is multiplied by $T_1^{\oplus}[42]$ and $T_1^{\oplus}[44]$; $T_1^{\oplus}[48]$ is multiplied by $T_1^{\oplus}[47]$ and $T_1^{\oplus}[49]$; and $T_1^{\oplus}[56]$ is multiplied by $T_1^{\oplus}[55]$ and $T_1^{\oplus}[57]$. The ANFs of $T_1^{\oplus}[42]$, $T_1^{\oplus}[44]$, $T_1^{\oplus}[47]$, $T_1^{\oplus}[49]$, $T_1^{\oplus}[55]$, and $T_1^{\oplus}[57]$ are as follows:

$$\begin{aligned} T_1^{\oplus}[42] &= T_0^{\dagger}[24] + K_1[24] + T_0^{\dagger}[48] + K_1[48] + T_0^{\dagger}[63] + K_1[63], \\ T_1^{\oplus}[44] &= T_0^{\dagger}[5] + K_1[5] + T_0^{\dagger}[30] + K_1[30] + T_0^{\dagger}[54] + K_1[54], \\ T_1^{\oplus}[47] &= T_0^{\dagger}[14] + K_1[14] + T_0^{\dagger}[39] + K_1[39] + T_0^{\dagger}[63] + K_1[63], \\ T_1^{\oplus}[49] &= T_0^{\dagger}[5] + K_1[5] + T_0^{\dagger}[20] + K_1[20] + T_0^{\dagger}[45] + K_1[45], \\ T_1^{\oplus}[55] &= T_0^{\dagger}[23] + K_1[23] + T_0^{\dagger}[38] + K_1[38] + T_0^{\dagger}[63] + K_1[63], \\ T_1^{\oplus}[57] &= T_0^{\dagger}[5] + K_1[5] + T_0^{\dagger}[29] + K_1[29] + T_0^{\dagger}[44] + K_1[44]. \end{aligned}$$

Excluding the bits of K_1, the monomial $T_0[3]T_0[4]$ is multiplied by the following terms:

$$\begin{aligned} & T_0^{\dagger}[5], T_0^{\dagger}[14], T_0^{\dagger}[20], T_0^{\dagger}[23], T_0^{\dagger}[24], T_0^{\dagger}[29], T_0^{\dagger}[30], \\ & T_0^{\dagger}[38], T_0^{\dagger}[39], T_0^{\dagger}[44], T_0^{\dagger}[45], T_0^{\dagger}[48], T_0^{\dagger}[54], T_0^{\dagger}[63]. \end{aligned}$$



(a) Growth of the degree in Cube-I

(b) Growth of the degree in Cube-II

Figure 3: Comparison of Cube-I and Cube-II in 6-round ChiLow-32

According to Cube-II (Equation 2) and the ANFs of χ_{64}, the following terms involve cube variables, while the remaining terms are constants:

$$\begin{aligned}
T_0^\dagger[5] &= T_0[5] + K_0[5] + (T_0[6] + K_0[6] + 1)(T_0[7] + K_0[7]),\\
T_0^\dagger[14] &= T_0[14] + K_0[14] + (T_0[15] + K_0[15] + 1)(T_0[16] + K_0[16]),\\
T_0^\dagger[20] &= T_0[20] + K_0[20] + (T_0[21] + K_0[21] + 1)(T_0[22] + K_0[22]),\\
T_0^\dagger[30] &= T_0[30] + K_0[30] + (T_0[0] + K_0[0] + 1)(T_0[1] + K_0[1]),\\
T_0^\dagger[45] &= T_0[45] + K_0[45] + (T_0[46] + K_0[46] + 1)(T_0[47] + K_0[47]),\\
T_0^\dagger[48] &= T_0[48] + K_0[48] + (T_0[49] + K_0[49] + 1)(T_0[50] + K_0[50]),\\
T_0^\dagger[54] &= T_0[54] + K_0[54] + (T_0[55] + K_0[55] + 1)(T_0[56] + K_0[56]).
\end{aligned}$$

After the χ_{64} in the second round, seven degree-3 monomials will appear (which is contained by the ■ bit in $T_1^\dagger$ in Figure 3b) as follows:

$$\begin{aligned}
&(T_0[7] + K_0[7])\,T_0[3]T_0[4]T_0[6], \quad (T_0[16] + K_0[16])\,T_0[3]T_0[4]T_0[15]\\
&(T_0[22] + K_0[22])\,T_0[3]T_0[4]T_0[21], \quad (T_0[0] + K_0[0] + 1)\,T_0[3]T_0[4]T_0[1]\\
&(T_0[46] + K_0[46] + 1)\,T_0[3]T_0[4]T_0[47], \quad (T_0[50] + K_0[50])\,T_0[3]T_0[4]T_0[49]\\
&(T_0[56] + K_0[56])\,T_0[3]T_0[4]T_0[55]
\end{aligned}$$

Fix phase. These degree-3 monomials necessarily contribute to each term of the superpolys of Cube-II, since the degree-33 monomial is generated by multiplying one degree-3 monomial with other terms. Consequently, the superpoly of Cube-II in the i-th output bit can be expressed as

$$\begin{aligned}
&(T_0[7] + K_0[7]) \cdot f_0[i] + (T_0[16] + K_0[16]) \cdot f_1[i] + (T_0[22] + K_0[22]) \cdot f_2[i]\\
&+ (T_0[0] + K_0[0] + 1) \cdot f_3[i] + (T_0[46] + K_0[46] + 1) \cdot f_4[i] + (T_0[50] + K_0[50]) \cdot f_5[i]\\
&+ (T_0[56] + K_0[56]) \cdot f_6[i]
\end{aligned}$$

where $f_0[i], f_1[i], \ldots, f_6[i]$ denote complex polynomials that depend on the secret key and correspond to the i-th output bit. If we impose the key conditions that all degree-3 monomials are cancelled, the cube sums in all output bits must be 0, which allow us to recover the seven key bits. The key recovery algorithm is illustrated in Algorithm 2.

In Algorithm 2, the overall complexity is bounded by 2^{40} in both data and time (with the data complexity limited to 2^{40}). Once the seven key bits have been recovered, we can designate $T_0[16]$ as a cube variable and fix $T_0[15]$ as a constant. This choice yields a degree-3 monomial,

$$(T_0[15] + K_0[15] + 1)\,T_0[3]T_0[4]T_0[16],$$

in the second round, which enables the recovery of $K_0[15]$. Noting that Algorithm 2 already employs the required data, and therefore, this step does not increase the data complexity. The remaining 120 key bits can subsequently be determined through exhaustive search.

Assumption. It is important to note that Algorithm 2 assumes that when the seven key conditions are not satisfied, it is impossible for all cube sums corresponding to Cube-II to be zero. Unlike the 5-round case, verifying this assumption is not practical due to the high dimension of Cube-II and the involvement of many key bits in their superpolys. There are $2^7 - 1$ cases where the seven key conditions are not satisfied. In our experiments, we tested the bias of 32 superpolys under each case and confirmed that the superpolys are nearly balanced. Specifically, for each superpoly, approximately 5 out of 10 tests evaluate it to 0. Assuming that the 32 superpolys are independent, the probability that all cube

Algorithm 2: Recovery of seven key bits for 6-round ChiLow

Input: Oracle of ChiLow-32

Output: Values of $K_0[7]$, $K_0[16]$, $K_0[22]$, $K_0[0]$, $K_0[46]$, $K_0[50]$, $K_0[56]$

```
Initialize the key variables K_0[7], K_0[16], K_0[22], K_0[0], K_0[46], K_0[50], K_0[56];
for T_0[7], T_0[16], T_0[22], T_0[0], T_0[46], T_0[50], T_0[56] do
    Compute the cube sums of Cube-II for all output bits;
    if all cube sums are equal to 0 then
        K_0[7] ⟵ T_0[7];
        K_0[16] ⟵ T_0[16];
        K_0[22] ⟵ T_0[22];
        K_0[0] ⟵ T_0[0] + 1;
        K_0[46] ⟵ T_0[46] + 1;
        K_0[50] ⟵ T_0[50];
        K_0[56] ⟵ T_0[56];
        return K_0[7], K_0[16], K_0[22], K_0[0], K_0[46], K_0[50], K_0[56];
    end
    else
        Continue;
    end
end
```

sums are zero when the seven key conditions are not satisfied is $(1/2)^{32}$, which is almost zero. Therefore, our conditional cube attack on the 6-round cipher is valid.

Remark. The number of degree-3 monomials in the second round (after the χ function) determines both the time and data complexities of the attack. Fewer degree-3 monomials generally result in lower complexity. In our attack, we exhaustively test all possible pairs of adjacent cube variables and find that at least seven degree-3 monomials inevitably appear in the second round. While there exist multiple choices of adjacent cube variable pairs that yield seven degree-3 monomials, in this paper we select $T_0[3]$ and $T_0[4]$ for our analysis.

Complexity. The attack consists of two main phases. The first is the conditional cube attack, which requires 2^{40} operations. The second is the exhaustive search, which requires 2^{120} operations. Therefore, the overall time complexity of the 6-round attack is 2^{120}. The data complexity is 2^{40}, while the memory complexity is negligible.

5 Borderline Cube Attack on ChiLow-32

5.1 Practical Attack on 5-round ChiLow-32

The key recovery process for 5-round ChiLow-32 based on the borderline cube consists of two steps. In the offline phase, the superpolys corresponding to the selected borderline cube are recovered. In the online phase, the ChiLow-32 oracle is queried to compute the cube sum, which is then used to recover the secret key

Cube configuration. In this attack, we employ 16 non-adjacent cube variables to perform key recovery. Without loss of generality, we select these 16 cube variables from the tweak path, specified as

$$\text{Tweak path: } \{32, 34, 36, 38, 40, 42, 44, 46, 48, 50, 52, 54, 56, 58, 60, 62\}.$$

In the first round, these cube variables multiply with certain key bits. For example, $T_0[32]$ multiplies with $K_0[31]$ and $K_0[33]$, $T_0[34]$ multiplies with $K_0[33]$ and $K_0[35]$, and so

on. After removing duplicate key bits, in total 17 key bits are involved in the superpoly corresponding to this borderline cube, as listed below:

$$K_0[31], K_0[33], K_0[35], K_0[37], K_0[39], K_0[41], K_0[43], K_0[45],$$
$$K_0[47], K_0[49], K_0[51], K_0[53], K_0[55], K_0[57], K_0[59], K_0[61], K_0[63].$$

Offline phase: superpoly recovery. To compute the superpolys of all output bits of 5-round ChiLow-32, we employ symbolic computation. Specifically, we first compute the ANFs of the state after 4 rounds, namely $X_4^{\oplus}$. For each bit, we store all degree-8 monomials in a hash table, where the monomials serve as keys and their coefficients as values. These hash tables require at most $32 \cdot \binom{16}{8} \cdot 2^{17} \cdot \approx 2^{39.65}$ bits of memory. However, in oue experiments, the coefficient of degree-8 monomial contains at most 2^{11} terms. Thus, These hash tables require $32 \cdot \binom{16}{8} \cdot 2^{11} \cdot \approx 2^{33.65}$ bits of memory.

When computing the superpolys of the 5-round output bits, it suffices to consider only the products of degree-8 terms from two bits. For example, consider $X_5[0]$, whose ANF is given by

$$X_5[0] = X_4^{\oplus}[0] + (X_4^{\oplus}[1] + 1)X_4^{\oplus}[2].$$

It is clear that the coefficients of the resulting degree-16 monomials are generated by multiplying the degree-8 monomials in $X_4^{\oplus}[1]$ with the degree-8 monomials in $X_4^{\oplus}[2]$.

Concretely, for each degree-8 term in the first bit's hash table, we check whether there exists a corresponding degree-8 monomial in the second bit's hash table such that their product yields a degree-16 monomial. If such a match exists, we multiply their coefficients and store the result. After iterating through all entries in the hash tables, the coefficients of the resulting superpoly are obtained.

With the 32 recovered superpolys, we can define a vectorial Boolean function $F : \mathbb{F}_2^{17} \to \mathbb{F}_2^{32}$ that maps the key bits $(K_0[31], K_0[33], \ldots, K_0[63])$ to the corresponding 32 cube sums. To facilitate key recovery, we then store each full key candidate $(K_0[31], K_0[33], \ldots, K_0[63]) \in \mathbb{F}_2^{17}$ in a hash table $\mathbb{H}$ indexed by $F(K_0[31], K_0[33], \ldots, K_0[63])$. This requires approximately $2^{17} \times 32 = 2^{22}$ bits of memory.

Online phase: key recovery. In the online phase, we query the ChiLow-32 oracle to obtain the cube sums for the 32-bit output of the 5-round cipher. For convenience, the cube sum is denoted as $(z_0, z_1, \ldots, z_{31})$. The corresponding key candidate can then be directly retrieved from the hash table $\mathbb{H}[(z_0, z_1, \ldots, z_{31})]$. Since the 32-bit superpolys depend on only 17 key bits, this procedure yields a unique key candidate. In this complexity, the time complexity is 2^{16} queries of ChiLow-32.

Recover the remaining key bits. Under one selected borderline cube of dimension 16, we can recover the values of 17 key bits. By using other borderline cubes, additional key bits can be obtained. Employing six 16-dimensional borderline cubes allows us to recover 96 key bits, specifically $K_0[i]$ for $0 \le i \le 63$ and $96 \le i \le 127$. The remaining key bits, $K_0[i]$ for $64 \le i \le 95$, which are XORed into D'_{32} prior to the first round, can be recovered via exhaustive search.

Complexity. The time complexity of the attack can be divided into three components. The first component corresponds to the offline phase for recovering the superpolys, which, in our experiments, can be completed within a few minutes. The second component corresponds to the online phase: for a single borderline cube, 2^{16} queries are required, and for six cubes, a total of $6 \cdot 2^{16}$ queries are needed. The third component involves an exhaustive search of the remaining key bits, which requires 2^{32} queries. Therefore, the overall time complexity is dominated by 2^{32} queries.

The data complexity is $6 \cdot 2^{16} \approx 2^{18.58}$. The memory complexity is mainly determined by the hash tables, which requires approximately $2^{33.65}$ bits of memory.

5.2 Practical Attack on 6-round ChiLow-32

Cube configuration. For the 6-round ChiLow-32, we select a 32-dimensional cube in which all cube variables are non-adjacent. An example of such a cube configuration is given below:

$$\begin{aligned}\text{Data path: } &\{1\},\\ \text{Tweak path: } &\{1,3,5,7,9,11,13,15,17,19,21,23,25,27,29\},\\ &\{32,34,36,38,40,42,44,46,48,50,52,54,56,58,60,62\}.\end{aligned}$$

In the first round, the following 35 key bits are multiplied by the cube variables and are also involved in the superpolys.

$$\begin{aligned}&K_0[96], K_0[98], K_0[0], K_0[2], K_0[4], K_0[6], K_0[8], K_0[10], K_0[12], K_0[14],\\ &K_0[16], K_0[18], K_0[20], K_0[22], K_0[24], K_0[26], K_0[28], K_0[30],\\ &K_0[31], K_0[33], K_0[35], K_0[37], K_0[39], K_0[41], K_0[43], K_0[45],\\ &K_0[47], K_0[49], K_0[51], K_0[53], K_0[55], K_0[57], K_0[59], K_0[61], K_0[63].\end{aligned}$$

Offline phase: superpoly recovery. Since the 32-dimensional borderline cube only involves 35 key bits in its superpolys, according to Lemma 4, it takes 2^{32+35} evaluations to recover the superpolys. The time complexity is reduced by using symbolic computation.

We first compute the coefficients of all degree-16 monomials for 32 bits state after 5 rounds, namely $X_5^{\oplus}$. For each bit, we store $\binom{32}{16}$ degree-16 monomials in a hash table, where the monomials serve as keys and their coefficients as values. on theory, these hash tables require at most $32 \cdot \binom{32}{16} \cdot 2^{35} \cdot 35 \approx 2^{74.28}$ bits of memory. However, In our experiments, we consider the best-case scenario in which a degree-16 monomial involves only 17 key bits. In this case, the coefficient of degree-16 monomial contains about 2^{11} key monomials. For the worst-case scenario, where a degree-16 monomial involves the maximum number of key bits (32 key bits), the number of terms with respect to the 32-bit key increases to approximately 2^{15}. Consequently, the corresponding hash tables require approximately $32 \cdot \binom{32}{16} \cdot 2^{15} \cdot 35 \approx 2^{54.28}$ bits of memory.

When computing the superpolys of the 6-round output bits, it suffices to consider only the products of degree-16 terms from two bits. For example, consider $X_6[0]$, whose ANF is given by

$$X_6[0] = X_5^{\oplus}[0] + (X_5^{\oplus}[1] + 1)X_5^{\oplus}[2]$$

The coefficients of the resulting degree-32 monomials are obtained by multiplying the degree-16 monomials in $X_5^{\oplus}[1]$ with those in $X_5^{\oplus}[2]$. For each degree-16 term in the first hash table, we check for a matching term in the second hash table whose product gives a degree-32 monomial. If a match exists, we multiply their coefficients and store the result. After processing all entries, the coefficients of the superpoly are obtained. Treating each multiplication as a single query of ChiLow-32, this step requires $\binom{32}{16} \approx 2^{29.16}$ queries.

With the 32 recovered superpolys, we define a vectorial Boolean function $F : \mathbb{F}_2^{35} \to \mathbb{F}_2^{32}$ that maps the key bits $(K_0[96], K_0[98], \ldots, K_0[63])$ to the corresponding 32 cube sums. Similar to 5-round, we then store each full key candidate $(K_0[96], K_0[98], \ldots, K_0[63])$ in a hash table $\mathbb{H}$ indexed by $F((K_0[96], K_0[98], \ldots, K_0[63]))$. This requires approximately $2^{35} \times 32 = 2^{40}$ bits of memory.

Online phase: key recovery. In the online phase, we query the ChiLow oracle to obtain the 32-bit cube sums $(z_0, z_1, \ldots, z_{31})$ of the 5-round cipher. The corresponding key candidates can then be directly retrieved from the hash table $\mathbb{H}[(z_0, z_1, \ldots, z_{31})]$. Since the 32-bit superpolys depend on only 35 key bits, this yields 2^3 candidates, with a time complexity of 2^{32} oracle queries.

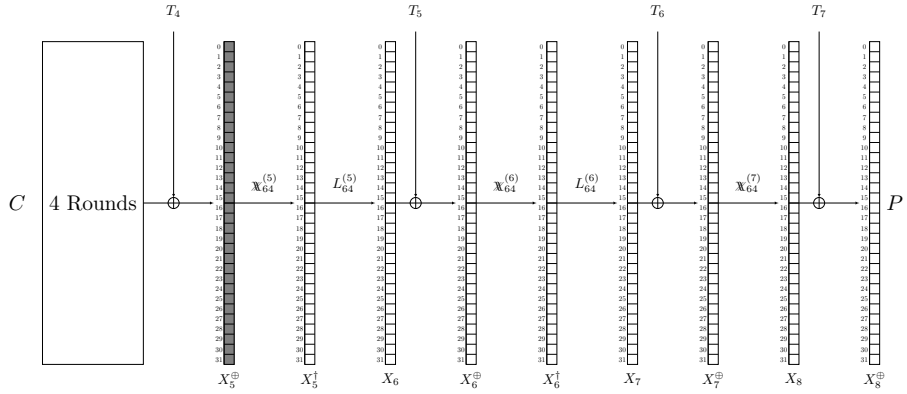


Figure 4: The illustration of recovering the round key for 7-round

Recover the remaining key bits. Using a single 32-dimensional borderline cube reduces the key space by 2^{32}. With three such cubes, we can recover 96 key bits, specifically $K_0[i]$ for $0 \leq i \leq 63$ and $96 \leq i \leq 127$. The remaining bits, $K_0[i]$ for $64 \leq i \leq 95$, which are XORed into D'_{32} before the first round, can then be recovered via exhaustive search.

Complexity. The time complexity of the attack can be divided into three components. The first component corresponds to the offline phase for recovering the superpolys, which is $2^29.16$. The second component corresponds to the online phase: for a single borderline cube, 2^{32} queries are required, and for three cubes, a total of $3 \cdot 2^{32}$ queries are needed. The third component involves an exhaustive search of the remaining key bits, which requires 2^{32} queries. Therefore, the overall time complexity is $2^{29.16} + 3 \cdot 2^{32} + 2^{32} \approx 2^{34}$.

The data complexity is $3 \cdot 2^{32} \approx 2^{33.58}$. The memory complexity is mainly determined by the hash tables, which require $2^{54.28}$ bits.

6 Integral Attack on 7-round ChiLow-32

In this section, we describe the round key recovery for 7-round ChiLow-32 and then discuss how to recover the master key. The attack leverages a 4-round borderline cube of dimension 8, to which three additional rounds are appended as shown in Figure 4. Our attack is conducted in the single-tweak setting, meaning that the cube variables are selected exclusively in the data path, as follows:

$$\text{Data path: } \{16, 18, 20, 22, 24, 26, 28, 30\}.$$

We encrypt 2^8 plaintexts to the ending of 4-th round(i.e., $X_5^{\oplus}$) to facilitate the key recovery. The key bits involved in the superpolys of the 8-dimensional borderline cube comprise 9 bits, as follows:

$$K_0[111], K_0[113], K_0[115], K_0[117], K_0[119], K_0[121], K_0[123], K_0[125], K_0[127].$$

For 2^8 plaintexts, guess $T_7[0:31]$, $T_6[0:31]$, $T_5[0:31]$, and compute

$$X_7^{\oplus} = {\mathbb{K}_{64}^{(7)}}^{-1}\big(X_8^{\oplus} + T_7[0:31]\big),$$

$$X_6^{\oplus} = {\mathbb{K}_{64}^{(6)}}^{-1}\Big({L_{64}^{(6)}}^{-1}\big(X_7^{\oplus} + T_6[0:31]\big)\Big),$$

$$X_5^{\oplus} = {\mathbb{K}_{64}^{(5)}}^{-1}\Big({L_{64}^{(5)}}^{-1}\big(X_6^{\oplus} + T_5[0:31]\big)\Big).$$

For each guess of $T_7[0:31]$, $T_6[0:31]$, and $T_5[0:31]$, we obtain 2^8 instances of $X_5^{\oplus}$, which allow us to compute the cube sums for the 8-dimensional borderline cube. This, in turn, enables the construction of an overdefined system comprising 32 equations in 9 key variables (the superpolys involve 9 key bits and can be recovered easily). If the system admits a solution, the corresponding guess is retained as a candidate; otherwise, it is discarded. On average, only one out of 2^{23} such systems possesses a solution, resulting in a reduction of the round key space from 2^{96} to 2^{73}. But we cannot utilize another 8-dimensional borderline cube to obtain additional information, since the security claim limits the data complexity to 2^8 under a single tweak,

In this attack, the time complexity amounts to 2^{104} look-up table operations, which can be viewed as 2^{104} queries of ChiLow-32. The data complexity is 2^8, and the memory complexity is negligible.

Discussion about recovering the master key. In the 7-round attack, the round key space can be reduced from 2^{96} to 2^{73} under a fixed tweak. However, recovering the master key from the round key remains challenging due to the nested tweak-key schedule in ChiLow. In ChiLow, the cipher state receives subkeys from the tweak state, while the tweak state, in turn, obtains subkeys from the key state. Moreover, the round key information maintains a complex relationship with the master key because of the rapid diffusion occurring in both the tweak and key states.

A straightforward approach is to enumerate all 2^{128} master key guesses, following the key and tweak paths to satisfy the conditions on $T_7[0:31]$, $T_6[0:31]$, and $T_5[0:31]$. On average, this reduces the master key space to 2^{105}, after which the remaining keys can be recovered via exhaustive search. However, the benefit of this method is minimal, corresponding to a reduction of 2^{128} operations along the data path compared with a brute-force attack. Since the data length is 32 bits, while the tweak schedule uses 64 bits and the key schedule 128 bits, the time complexity of recovering the master key can be estimated as

$$\frac{64+128}{32+64+128} \cdot 2^{128} \approx 2^{127.78},$$

which is nearly to the brute-force attack. Therefore, how to efficiently recover the master key from the available round key information remains an open problem.

7 Conclusion

In this paper, we present the first third-party cryptanalysis of ChiLow, a tweakable block cipher based on the low-degree nonlinear χ function. We develop three types of cube-based attacks—conditional cube attack, borderline cube attack, and integral attack—under both single-tweak and multi-tweak settings. Our results extend previous key recovery analyses from 4 rounds to 7 rounds, demonstrating the effectiveness of cube-like attacks on reduced-round ChiLow. Although the full 8-round cipher remains secure under our analysis, the proposed attacks provide valuable insights into its structural properties and highlight potential avenues for further cryptanalysis. Further work is required to explore efficient methods for recovering the master key when information about the round keys is available. Additionally, analyzing the full-round ChiLow or designing variants resistant to cube-like attacks remains an important direction for future research.

References

[BDG+25] Yanis Belkheyar, Patrick Derbez, Shibam Ghosh, Gregor Leander, Silvia Mella, Léo Perrin, Shahram Rasoolzadeh, Lukas Stennes, Siwei Sun, Gilles Van Assche, and Damian Vizár. Chilow and chichi: New constructions for code

encryption. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part I*, volume 15601 of *Lecture Notes in Computer Science*, pages 212–243. Springer, 2025.

[Can16] Anne Canteaut. Lecture notes on cryptographic boolean functions. *Inria, Paris, France*, 3, 2016.

[CCH10] Claude Carlet, Yves Crama, and Peter L Hammer. Boolean functions for cryptography and error-correcting codes., 2010.

[DEMS21] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläffer. Ascon v1.2: Lightweight authenticated encryption and hashing. *J. Cryptol.*, 34(3):33, 2021.

[DLWQ17] Xiaoyang Dong, Zheng Li, Xiaoyun Wang, and Ling Qin. Cube-like attack on round-reduced initialization of ketje sr. *IACR Trans. Symmetric Cryptol.*, 2017(1):259–280, 2017.

[DMP+15] Itai Dinur, Pawel Morawiecki, Josef Pieprzyk, Marian Srebrny, and Michal Straus. Cube attacks and cube-attack-like cryptanalysis on the round-reduced keccak sponge function. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part I*, volume 9056 of *Lecture Notes in Computer Science*, pages 733–761. Springer, 2015.

[DR20] Joan Daemen and Vincent Rijmen. *The Design of Rijndael - The Advanced Encryption Standard (AES), Second Edition.* Information Security and Cryptography. Springer, 2020.

[DS09] Itai Dinur and Adi Shamir. Cube attacks on tweakable black box polynomials. In Antoine Joux, editor, *Advances in Cryptology - EUROCRYPT 2009, 28th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cologne, Germany, April 26-30, 2009. Proceedings*, volume 5479 of *Lecture Notes in Computer Science*, pages 278–299. Springer, 2009.

[HLLT20] Phil Hebborn, Baptiste Lambin, Gregor Leander, and Yosuke Todo. Lower bounds on the degree of block ciphers. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part I*, volume 12491 of *Lecture Notes in Computer Science*, pages 537–566. Springer, 2020.

[HLM+20] Yonglin Hao, Gregor Leander, Willi Meier, Yosuke Todo, and Qingju Wang. Modeling for three-subset division property without unknown subset - improved cube attacks against trivium and grain-128aead. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part I*, volume 12105 of *Lecture Notes in Computer Science*, pages 466–495. Springer, 2020.

[HSWW20] Kai Hu, Siwei Sun, Meiqin Wang, and Qingju Wang. An algebraic formulation of the division property: Revisiting degree evaluations, cube attacks, and key-independent sums. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part I*, volume 12491 of *Lecture Notes in Computer Science*, pages 446–476. Springer, 2020.

[Hu24] Kai Hu. Improved conditional cube attacks on ascon aeads in nonce-respecting settings with a break-fix strategy. *IACR Trans. Symmetric Cryptol.*, 2024(2):118–140, 2024.

[HWX+17] Senyang Huang, Xiaoyun Wang, Guangwu Xu, Meiqin Wang, and Jingyuan Zhao. Conditional cube attack on reduced-round keccak sponge function. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology - EUROCRYPT 2017 - 36th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Paris, France, April 30 - May 4, 2017, Proceedings, Part II*, volume 10211 of *Lecture Notes in Computer Science*, pages 259–288, 2017.

[LBDW17] Zheng Li, Wenquan Bi, Xiaoyang Dong, and Xiaoyun Wang. Improved conditional cube attacks on keccak keyed modes with MILP method. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I*, volume 10624 of *Lecture Notes in Computer Science*, pages 99–127. Springer, 2017.

[LDW17] Zheng Li, Xiaoyang Dong, and Xiaoyun Wang. Conditional cube attack on round-reduced ASCON. *IACR Trans. Symmetric Cryptol.*, 2017(1):175–202, 2017.

[PHHW25] Shuo Peng, Kai Hu, Jiahui He, and Meiqin Wang. Improved key recovery attacks of ascon. In Arpita Patra, editor, *Topics in Cryptology - CT-RSA 2025 - Cryptographers' Track at the RSA Conference 2025, San Francisco, CA, USA, April 28-May 1, 2025, Proceedings*, volume 15598 of *Lecture Notes in Computer Science*, pages 123–146. Springer, 2025.

[RHSS21] Raghvendra Rohit, Kai Hu, Sumanta Sarkar, and Siwei Sun. Misuse-free key-recovery and distinguishing attacks on 7-round ascon. *IACR Trans. Symmetric Cryptol.*, 2021(1):130–155, 2021.

[SG18] Ling Song and Jian Guo. Cube-attack-like cryptanalysis of round-reduced keccak using MILP. *IACR Trans. Symmetric Cryptol.*, 2018(3):182–214, 2018.

[SGSL18] Ling Song, Jian Guo, Danping Shi, and San Ling. New MILP modeling: Improved conditional cube attacks on keccak-based constructions. In Thomas Peyrin and Steven D. Galbraith, editors, *Advances in Cryptology - ASIACRYPT 2018 - 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2-6, 2018, Proceedings, Part II*, volume 11273 of *Lecture Notes in Computer Science*, pages 65–95. Springer, 2018.

[TIHM17] Yosuke Todo, Takanori Isobe, Yonglin Hao, and Willi Meier. Cube attacks on non-blackbox polynomials based on division property. In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology - CRYPTO 2017 -*

37th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 20-24, 2017, Proceedings, Part III, volume 10403 of *Lecture Notes in Computer Science*, pages 250–279. Springer, 2017.

[TM16] Yosuke Todo and Masakatu Morii. Bit-based division property and application to simon family. In Thomas Peyrin, editor, *Fast Software Encryption - 23rd International Conference, FSE 2016, Bochum, Germany, March 20-23, 2016, Revised Selected Papers*, volume 9783 of *Lecture Notes in Computer Science*, pages 357–377. Springer, 2016.

[Tod15] Yosuke Todo. Structural evaluation by generalized integral property. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part I*, volume 9056 of *Lecture Notes in Computer Science*, pages 287–314. Springer, 2015.

[WHG+19] Senpeng Wang, Bin Hu, Jie Guan, Kai Zhang, and Tairong Shi. Milp-aided method of searching division property using three subsets and applications. In Steven D. Galbraith and Shiho Moriai, editors, *Advances in Cryptology - ASIACRYPT 2019 - 25th International Conference on the Theory and Application of Cryptology and Information Security, Kobe, Japan, December 8-12, 2019, Proceedings, Part III*, volume 11923 of *Lecture Notes in Computer Science*, pages 398–427. Springer, 2019.

[WHT+18] Qingju Wang, Yonglin Hao, Yosuke Todo, Chaoyun Li, Takanori Isobe, and Willi Meier. Improved division property based cube attacks exploiting algebraic properties of superpoly. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2018, Proceedings, Part I*, volume 10991 of *Lecture Notes in Computer Science*, pages 275–305. Springer, 2018.

[XZBL16] Zejun Xiang, Wentao Zhang, Zhenzhen Bao, and Dongdai Lin. Applying MILP method to searching integral distinguishers based on division property for 6 lightweight block ciphers. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology - ASIACRYPT 2016 - 22nd International Conference on the Theory and Application of Cryptology and Information Security, Hanoi, Vietnam, December 4-8, 2016, Proceedings, Part I*, volume 10031 of *Lecture Notes in Computer Science*, pages 648–678, 2016.